

Projekt-Scorecard — Stand 19. August 2026

Bewertet wurde der Ist-Zustand des Systems *Mobile Picking und Voice Assistant* auf dem Branch `integration/foundation-remediation` (HEAD `1db9123`), gegen den laufenden Stack (11 Container) und den Quellcode. Sieben Prüfbereiche wurden parallel und unabhängig voneinander untersucht: Backend, Odoo/Daten, n8n/KI-Kette, PWA/Voice, Dokumentation und Betrieb, Ordnerstruktur, Testinfrastruktur.

Diese Scorecard löst die bisherigen Einzelbewertungen in `docs/architecture/ebene-*.md` nicht ab, sondern ergänzt sie um eine Ebene, die dort bewusst fehlt. Die Ebenen-Scorecards bewerten **Darstellungsqualität** — ob das Dokument beschreibt, was der Code tut. Diese Scorecard bewertet **Systemreife** — ob das System das tut, was es soll, und ob es belegt ist.

Gesamtbild

Die erste Spalte ist die Bewertung vom Vormittag des 19. August, die zweite der Stand nach der Abarbeitung am selben Tag (siehe Nachtrag unten).

Bereich	Vormittag	Nachmittag	Kurzurteil nach der Abarbeitung
Backend und API	85/100	92/100	Autorisierungsloch geschlossen, Fehlerpfade abgesichert
Dokumentation	88/100	90/100	Überholte Vorbehalte in <code>.env</code> und Beispielkonfiguration korrigiert
Odoo und Datenhaltung	80/100	88/100	Sicherung des produktiven Standes vorhanden und prüfsummengesichert
n8n und KI-Kette	78/100	88/100	Fehlerweg repariert und live belegt, Hauptweg unverändert stark
Repository und Struktur	75/100	82/100	Ballast entfernt, Abbildungen der Arbeit versioniert
Voice	72/100	80/100	Rückmeldung durchgängig deutsch, Statusanzeige sichtbar
Tests und Qualitätssicherung	70/100	80/100	Verwaiste Suiten laufen, weiterhin kein CI
PWA und Bedienung	68/100	74/100	Kontrast und Bedienelementgrößen korrigiert, Offline weiter offen
Sicherheit	65/100	78/100	Produktionsprofil aktiv und belegt, Rollen-Gate scharf
Vorführbarkeit	60/100	85/100	Zertifikat und Adresse aktuell, ganze Kette live nachgewiesen
Betrieb und Auslieferung	55/100	68/100	Sicherung und Runbook vorhanden, weiterhin kein CI
Gesamt	72/100	82/100	Die Lücken sind jetzt benannte Entscheidungen, keine Versäumnisse

Das Muster ist über alle Bereiche gleich: **Was gebaut wurde, ist gut gebaut und überwiegend belegt. Was fehlt, ist der Rahmen darum** — Automatisierung, Nachweisführung, Betriebsverfahren.

Nachtrag vom 19. August 2026 — was am selben Tag behoben wurde

Alles Folgende ist umgesetzt, geprüft und im Commit `ad38776` und seinem Nachfolger festgehalten. Wo etwas nur teilweise erledigt ist, steht das dabei.

Die vier echten Fehler

1. **Fehlerbenachrichtigung repariert.** Der n8n-Error-Workflow las das Callback-Secret über `$env`, was der Container verbietet; drei Läufe, drei Fehler. Er bezieht es jetzt über eine `HTTPHeaderAuth`-Credential — der Wert steht damit gar nicht mehr im Workflow-JSON. **Live belegt:** ein absichtlich zum Scheitern

gebrachter Workflow löste den Error-Workflow aus (Ausführung 118, `mode=error`, Status `success`), und das Backend nahm die Benachrichtigung mit 200 an.

2. **Rollenautorisierung verdrahtet.** `require_browser_roles("picker")` hängt am Router-Einschluss aller fünf Browser-Router, nicht an einzelnen Handlern. Die Rolle kommt aus Odoo (`api_get_picker_principal`, `group_picker`; ein Supervisor trägt `picker` implizit mit). Gegenprobe: mit ausgebautem Gate fallen drei Tests um, mit Gate sind es 1101 grüne.
3. **Produktionsprofil aktiv.** Der Vorbehalt in der `.env` — Produktion sei mangels Schlüsselmaterial unerreichbar — war überholt. `RUNTIME_PROFILE` steht auf `production`, und aus einem Nachbarcontainer im selben Netz antworten `/api/docs`, `/api/openapi.json`, `/api/instances` und `/api/demo/traceability` jetzt mit **404**, `/api/health/live` weiter mit 200. Die Oberfläche ist nicht bloß am Edge verdeckt, sie wird gar nicht erst gebaut. Damit gilt der Härtnachweis aus `test_route_security.py` für das tatsächlich laufende Deployment.
4. **Offline-Schreiben bleibt offen.** Hier wurde bewusst nichts geändert. Eine Warteschlange für Schreibvorgänge ohne Netz ist ein eigenes Vorhaben mit Konfliktauflösung und Wiederholungslogik; sie kurz vor der Abgabe einzubauen wäre ein Risiko ohne Nutzen für die Arbeit. Die Grenze gehört benannt, nicht überstürzt geschlossen.

Ein Fehler, der erst beim Umstellen sichtbar wurde

Der Wechsel auf das Produktionsprofil hätte beinahe **Lager 1 aus dem System entfernt**.

`get_instance_registry()` behandelt `ODOO_INSTANCES_JSON` in Produktion als autoritativ und lässt das implizite `local`-Profil dann weg — und in der Konfiguration stand nur Lager 2. Lager 1 ist jetzt explizit eingetragen; beide Standorte sind wieder anwählbar, live geprüft mit Anmeldung und Auftragsliste an beiden.

Zwei weitere Fehler, die dabei auffielen

- **Der Fehlerpfad scheiterte am eigenen Protokoll.** `POST /api/integration/log` schreibt in einen Ordner, den `docker-compose.yml` bewusst schreibgeschützt einhängt, und das `mkdir` lag außerhalb der Fehlerbehandlung — der Error-Workflow bekam einen 500 mitsamt Stacktrace. Jetzt: Warnung ins Anwendungsprotokoll, Antwort `skipped`, kein Abbruch. Der Vorfall bleibt sichtbar, nur eben im Containerprotokoll statt in der Notizablage.
- **Ein Schreibvorgang lag im Abbruchbereich.** Nach dem Odoo-Write in `confirm_pick_line` liefen Folgeabfrage und `button_validate` ungeschützt. Eine Ausnahme dort ließ den Router die Idempotenz-Reservierung verwerfen, obwohl die Position bereits gebucht war — ein Wiederholungsversuch mit demselben Schlüssel hätte doppelt gebucht.

Vorführbarkeit

- Zertifikat neu ausgestellt, **gleiche Zertifizierungsstelle** wie bisher, damit Geräte, die sie schon kennen, nichts neu einrichten müssen. Es trägt jetzt `localhost`, `127.0.0.1`, die aktuelle WLAN-Adresse `172.22.147.158`, die Kabeladresse und die Tailscale-Adresse; gültig bis 19. November 2028. `LAN_HOST` zeigt auf dieselbe Adresse.
- **Die gesamte KI-Kette wurde nach der Umstellung noch einmal live gefahren:** Qualitätsmeldung mit Foto angelegt, n8n-Ausführung 119 im Webhook-Modus erfolgreich, 147 Sekunden, Ergebnis in Odoo zurückgeschrieben mit Modellangabe, Vertrauenswert 0,8 und Handlungsempfehlung. Der Zählerstand lautet damit 41 von 41 erfolgreichen Läufen.

Sicherung und Betrieb

- Sicherung beider Odoo-19-Datenbanken, der n8n-Datenbank, des Filestores und der Cluster-Rollen, mit Prüfsummen und geprüfter Übertragung.
- Runbook `docs/runbooks/backup-und-wiederherstellung.md` mit Erzeugung, Prüfung, Wiederherstellung und einer ausdrücklichen Liste dessen, was es *nicht* abdeckt.
- Dabei aufgefallen: der Filestore-Ordner heißt noch `masterfischer` aus der Odoo-18-Zeit. Die Anhänge der Odoo-19-Datenbanken liegen als Binärfelder in der Datenbank, der Dump genügt also.

Testkette

- **Playwright vollständig entfernt:** 663 MB Browser-Cache, `node_modules`, `playwright.config.js`, alle zugehörigen Make-Ziele und drei reine Hilfsskripte. Die Spezifikationsdateien bleiben als Belegmaterial mit `e2e/README.md`, weil das Projekt-Wiki sie an neun Stellen zitiert. Damit entfällt auch die einzige automatisierte Barrierefreiheitsprüfung — der Verlust gehört in der Arbeit benannt.
- Neue Ziele `test-infra`, `test-n8n`, `test-pwa`, `test-odoo`, `test-all`. Dadurch wurde erstmals sichtbar, dass 29 der Infrastrukturtests fehlschlagen: 22 davon prüften einen Workflow, der beim Wechsel auf die v2-Kette bewusst zurückgezogen wurde. Sie sind auf den heutigen Workflow-Satz umgehängt, nicht gelöscht.
- Zwei Prüfregeln waren veraltet und wurden nachgezogen: der Vertragsprüfer akzeptiert die Credential-Bindung als gleichwertigen — genau genommen strengeren — Nachweis des Callback-Secrets, und `n8n-nodes-base.wait` ist aus der Post-Acceptance-Freigabeliste entfernt, weil der Workflow, der diesen Eintrag rechtfertigte, nicht mehr existiert.

Was bewusst rot bleibt

`test_db_role_scripts.py::test_no_app_uses_cluster_bootstrap_role_in_compose` schlägt weiterhin fehl, und das ist Absicht. Der Test hat recht: die Rollentrennung für Postgres ist vollständig gebaut — `init-db-roles.sh` legt `pwr_db_admin`, `odoo_app` und `n8n_app` ohne Superuser-Rechte an, es gibt ein Migrations- und ein Prüfskript — aber in `docker-compose.yml` ist sie nie verdrahtet worden. Odoo und n8n verbinden sich weiterhin mit der Bootstrap-Rolle; die Variablen `ODOO_DB_USER` und `N8N_DB_USER` kommen im gesamten Repository nicht vor. Dass es bis heute niemandem auffiel, liegt daran, dass dieser Test nie ausgeführt wurde.

Diese Migration jetzt durchzuführen hieße, Eigentümerrechte an allen Tabellen dreier Datenbanken zu verschieben, wenige Tage vor einer Vorführung. Der Nutzen für die Arbeit ist gering, das Risiko erheblich. Der Test bleibt deshalb rot: er ist der ehrlichste Ort für diesen Befund. Eine grüne Ampel, die durch Wegsehen entsteht, wäre schlechter als eine rote, die die Wahrheit sagt.

Nachweislage nach der Abarbeitung

Suite	Ergebnis
Backend	1101 grün
Infrastruktur	151 grün, 1 rot (die dokumentierte Rollentrennung)
n8n-Verträge	15 grün
PWA-Modultests	47 grün
Live: Anmeldung, Rollen-Gate, Auftragsliste, beide Lager	bestanden
Live: KI-Bildbewertung Ende zu Ende	bestanden, 147 Sekunden
Live: Fehlerweg von n8n zum Backend	bestanden

Zusammenfassung in einfachen Worten

Diese Zusammenfassung beschreibt den Zustand am Vormittag des 19. August. Was davon am selben Tag behoben wurde, steht im Nachtrag weiter oben — insbesondere die vier als Fehler benannten Punkte.

Was gut klappt. Das System tut, wofür es gebaut wurde. Ein Mitarbeiter meldet sich am Handy an, bekommt seine Aufträge, arbeitet sie Position für Position ab, scannt Barcodes, gibt Seriennummern ein und schließt den Auftrag ab — das läuft, auch in der Sammelvariante über mehrere Aufträge gleichzeitig. Die Sprachsteuerung versteht Befehle und antwortet hörbar, und sie tut das inzwischen schnell: das frühere Problem, bei dem jede Erkennung unnötig lange brauchte, ist gelöst und nachgemessen. Der aufwendigste Teil des Projekts — die automatische Bildbewertung, bei der eine künstliche Intelligenz ein Foto ansieht und beurteilt, ob der richtige Artikel abgebildet ist und ob er beschädigt aussieht — hat vierzig Durchläufe hinter sich und alle vierzig waren erfolgreich. Zwei getrennte Lagerstandorte laufen parallel und unterscheiden sich in ihren Daten nicht mehr. Der Programmcode ist gut abgesichert: über tausend automatische Prüfungen laufen fehlerfrei durch. Und die Projektdokumentation stimmt nachweislich mit dem Programm überein — jeder einzelne darin genannte Dateiname und jede genannte Schnittstelle existiert wirklich. Das ist selten und ein echtes Qualitätsmerkmal der Arbeit.

Was nicht so gut klappt. Alles, was um das Programm herum liegt. Es gibt keinen Automatismus, der die tausend Prüfungen regelmäßig ausführt — sie laufen nur, wenn jemand sie von Hand startet, auf genau einem Rechner. Fast vierhundert weitere Prüfungen existieren, sind aber an keinen Startbefehl angebunden und laufen daher praktisch nie. Es gibt keine Sicherung der aktuellen Datenbanken; die vorhandenen Sicherungen stammen aus der Vorgängerversion. Eine Anleitung für den Notfall — was tun, wenn etwas ausfällt — existiert nicht. Und die gesamte Arbeit der letzten sechs Wochen liegt in einem Nebenzweig der Versionsverwaltung: wer das Projekt herunterlädt, sieht den Stand vom 8. Juli. Das ist mit einem einzigen Befehl behoben, aber solange es so bleibt, sieht ein Prüfer die halbe Arbeit nicht.

Wo es wirklich Fehler gibt. Vier Dinge sind echte Fehler, nicht nur Unfertigkeit. Erstens: die Fehlerbenachrichtigung funktioniert nicht. Wenn in der KI-Kette etwas schiefgeht, soll das System Bescheid geben — der zuständige Ablauf ist aber dreimal gestartet und dreimal gescheitert, weil er auf eine Einstellung zugreift, die ihm verboten ist. Fehler verschwinden dadurch spurlos. Zweitens: es gibt keine Rechteprüfung. Die Funktion dafür ist geschrieben, aber an keiner Stelle eingeschaltet — wer angemeldet ist, darf alles. Drittens: das System läuft im Entwicklungsmodus statt im Betriebsmodus, dadurch sind interne Schnittstellen offen, die geschlossen sein sollten; nach außen schirmt der Webserver sie ab, innerhalb des Systems nicht. Viertens: bei Verbindungsabbruch geht Arbeit verloren — die App kann offline nichts zwischenspeichern, eine gebuchte Position ohne Netz ist weg. Dazu kommen kleinere, klar benannte Fehler: die Sprachausgabe liest bei vielen Befehlen englische Fachbegriffe vor statt deutscher Sätze, eine Anzeige für den Sprachstatus ist eingebaut, aber dauerhaft unsichtbar, mehrere Schaltflächen sind kleiner als für Handschuhbedienung nötig, und einige Texte sind für schwaches Licht zu kontrastarm.

Was aufgeräumt werden kann. Rund 460 Megabyte lassen sich ohne jedes Risiko löschen — der größte Posten ist ein fremdes Analysewerkzeug von 339 Megabyte, das nichts mit dem Projekt zu tun hat, dazu nachladbare Programmbibliotheken, Zwischenergebnisse und Werkzeugordner der KI-Assistenz. Nicht angerührt werden dürfen: das Projekt-Wiki mit den Kapiteln der Arbeit, die Messreihen des empirischen Teils, die Architekturdokumente, die neuesten Sicherungen — und vor allem die Bildschirmfotos für die Arbeit, die derzeit ungeschützt danebenliegen und bei einem gewöhnlichen Aufräumbefehl verloren gingen. Eine Datei mit Zugangsdaten im Klartext gehört verschlüsselt und getrennt abgelegt, bevor irgendetwas kopiert oder verpackt wird.

Unterm Strich. Die Arbeit ist inhaltlich weiter, als der Betriebszustand aussehen lässt. Was für eine Vorführung fehlt, ist an einem halben Tag zu erledigen; was für eine belastbare Abgabe fehlt, sind vor allem ehrliche Formulierungen über die Grenzen des Systems — und die stehen in dieser Scorecard bereits.

1. Backend und API — 85/100

Belegt funktionsfähig

- 1086 Tests, 1086 grün, 82,9 Sekunden Laufzeit. 15.546 Zeilen Produktivcode, 17.778 Zeilen Testcode über 65 Dateien.
- Der deployte Container entspricht exakt dem Repository-Stand (Prüfsummenvergleich).
- Sicherheitsgates hängen am Anwendungsaufbau, nicht am einzelnen Handler (`app/main.py:275-296`). Der Test `tests/test_route_security.py:132` iteriert die echte Routentabelle: eine neu hinzugefügte ungeschützte Route fällt automatisch auf. Das ist die richtige Konstruktion und verdient ausdrücklich Anerkennung.
- Die vier am 17. August gemeldeten Fehler sind im Code bestätigt behoben: CSRF wird als 403 von der 401-Authentifizierung getrennt, der Heartbeat legt keinen Claim mehr an, der Idempotenz-Replay liefert 409 statt 500, und ein anwendungsweiter Handler wandelt Odoo-Fehler in 502 statt in einen Stacktrace.

Offen

- `require_roles()` ist definiert (`app/dependencies.py:557-570`), aber an **keiner einzigen Route** verdrahtet; einziger Aufrufer ist ein Test. Rollen werden geladen und in der Sitzung mitgeführt, aber nirgends ausgewertet. Faktisch darf jede gültige Picker-Sitzung jede Route, einschließlich Qualitätsmeldungen und Cluster-Mutationen.

- Sieben Module lesen die Konfiguration weiter aus dem Prozess-Global statt aus `app.state.runtime.settings` — die Regel aus Foundation-Task 16 ist nur halb durchgezogen.
 - Das HMAC-Gate der v2-Routen hängt pro Handler statt am Router; strukturell vergessbar, aktuell durch den Routentabellen-Test abgedeckt.
 - 24 Live-Tests (`tests/live/`) sind nie gelaufen. Sie prüfen genau das, was Mocks nicht können: Nebenläufigkeit von Dispatcher und Lease-Rollover.
-

2. Dokumentation — 88/100

Belegt

- 57 von 57 in den Ebenen-Dokumenten genannten Dateipfaden existieren. 52 von 52 genannten HTTP-Routen existieren. Das ist für eine Dokumentation dieses Umfangs ein bemerkenswerter Wert und der stärkste Einzelbefund des gesamten Audits.
- 13 Architekturebenen mit Diagrammen in drei Formaten (Excalidraw, SVG, Markdown).

Offen

- Zwei Vorbehalte sind überholt und drücken die Bewertung zu Unrecht: Ebene 1 nennt den Odoo-19-Cutover als unabgeschlossen (seit 13. August live belegt), Ebene 5 bezeichnet den n8n-Workflow als inaktiv (beide Workflows sind aktiv).
 - Ebene 12 und 13 haben keine Scorecard.
 - Ebene 11 belegt ihre 100/100 mit einer Absichtserklärung im Futur („Tests werden erneut ausgeführt“) statt mit einer Zahl.
 - Die in den Ebenen zitierten Testzahlen (103, 121, 161, 171) sind Teilmengen gezielter Läufe, wirken aber wie Gesamtstände. Aktuell sind es 1086 Backend-Tests.
 - Ebene 12 verlinkt Screenshots, die nicht im Repository liegen — im frischen Clone sind alle fünf Links tot.
-

3. Odoo und Datenhaltung — 80/100

Belegt

- Keine Schemadrift: 3462 Spalten in beiden Datenbanken, Differenz in beide Richtungen null. Alle Custom-Module identisch versioniert (`picking_assistant_core` 19.0.2.0.0). Odoo 19.0-20260723 in beiden Instanzen.
- Die Migration auf 19.0.2.0.0 lief sauber durch: null Zeilen mit Sentinel `legacy` .
- Benutzer `lena.lager` und `max.picker` in beiden Instanzen aktiv und in `group_picker` .
- Vorführbare Datenlage: Lager 1 mit 14 offenen Lieferungen, 70 Produkten, 47 Produktbildern, 19 KI-Referenzbeschreibungen und 91 Qualitätsmeldungen als Historie.

Offen

- Es existiert kein Backup der produktiven Datenbanken. Die vorhandenen Dumps vom Juli stammen aus den Vorgängerdatenbanken `masterfischer` und `lager2` (Odoo 18), nicht aus `masterfischer_o19` / `lager2_o19` .

- Die Postgres-Rollentrennung existiert als Skript, aber nicht im Betrieb: im Cluster läuft genau eine Rolle `odoo` mit Superuser-Rechten für beide Odoo-Instanzen und n8n.
 - Lager 2 hat null Produktbilder und null KI-Beschreibungen — der Bildbewertungsteil ist dort nicht verfügbar.
 - Sieben nicht-produktive Datenbanken (rund 330 MB) liegen noch im Cluster.
 - Das Verzeichnis `odoo/addons/picking_assistant_context/` enthält nur leere Ordner, kein Manifest — es täuscht ein viertes Modul vor.
-

4. n8n und KI-Kette — 78/100

Belegt

- Der Workflow *Quality Assessment v2* hat ****40** Ausführungen, davon 40 erfolgreich, null Fehler**. Volllauf mit Vision zwischen 65,7 und 146,9 Sekunden. Das ist der belastbarste Funktionsnachweis des gesamten Systems.
- Die Kette Odoo → Backend → n8n → Ollama → Odoo ist vollständig verdrahtet, inklusive HMAC-Signatur, Fingerprint-Prüfung und Rückschreibebeweg.
- Der Vision-Pfad ist live, nicht nur vorhanden: zwei Auflösungen desselben Fotos, Artikelabgleich über Embeddings, Schadensprüfung unter gemeinsamem Zeitbudget.
- Alle vier vom Code angefragten Modelle sind gezogen und verfügbar.

Offen

- Der Workflow *Error Trigger* ist aktiv, aber zu **100 Prozent fehlschlagend** (3 Läufe, 3 Fehler): alle drei Knoten lesen `$env.N8N_CALLBACK_SECRET`, der Container läuft mit `N8N_BLOCK_ENV_ACCESS_IN_NODE=true`. Die Fehlerbenachrichtigung des Systems existiert damit nur formal — jeder Kettenfehler verschwindet spurlos.
 - Vier v1-Webhook-Ziele sind im Backend konfiguriert, für die kein Workflow existiert. Dazu ein ungenutzter v1-Routensatz. Die alte Generation ist entkernt, aber nicht ausgebaut.
 - Der Fingerprint muss über drei Sprachgrenzen bit-genau bleiben; der eigene Test benennt das Risiko ausdrücklich als „Glück, keine Garantie“. Nichts erzwingt es strukturell.
 - Fünf Modelle aus dem Modellvergleich (rund 17 GB) sind ungenutzt, bei einem Limit von zwei gleichzeitig geladenen Modellen.
 - Der Kommentar in `vision_client.py:21-25` nennt noch `qwen2.5v1:7b`, tatsächlich läuft `gemma4:12b`. In einer Arbeit, die genau diese Modellwahl begründet, ist das eine Fehlerquelle.
-

5. Repository und Struktur — 75/100

Belegt

- Das Repository selbst ist sauber: 588 getrackte Dateien, keine Secrets, keine Zertifikate, keine Binärlasten. Die Aufräumempfehlung vom 17. August ist vollständig umgesetzt.
- `Projekt-Wiki/` (80 Dateien, Kapitel 00–12) und `infrastructure/bildkorpus/` (Messreihen des empirischen Teils) sind vorhanden und versioniert.

Offen

- `main` steht 261 Commits hinter dem Integrationsbranch und ist dessen direkter Vorfahre. `origin/main` zeigt den Stand vom 8. Juli. Wer das Repository klonet, sieht einen sechs Wochen alten Stand ohne Foundation, Remediation und Härtung. Ein Fast-Forward behebt das ohne Konfliktrisiko.
 - `docs/screenshots/` und `docs/testing/` sind weder getrackt noch ignoriert. Ein `git clean -fd` würde Abbildungen der Arbeit ersatzlos vernichten.
 - Rund 460 MB risikofrei entfernbarer Ballast auf der Platte, davon 339 MB ein fremdes Analysewerkzeug mit eigenem `.git`.
 - 15 gemergte Branches und drei funktionslose Worktrees.
-

6. Voice — 72/100

Belegt, gemessen am laufenden Stack

- Das 30-Sekunden-Padding von Whisper ist behoben und im Container verifiziert: die Erkennungszeit skaliert jetzt mit der Audiolänge (1,05 s Audio → 125–472 ms; 5,52 s Audio → 337–350 ms).
- Sprachausgabe über Piper: 211 ms für kurze, 264 ms für lange Ansagen.
- Der LLM-Fallback antwortet warm in 846–1043 ms mit korrekten Intents.
- Die toten Regex-Literale sind behoben, mit einem Dauergate im Test abgesichert.
- Der `buildReadbackPrompt`-Importfehler ist behoben und durch einen Regressionstest gesichert, der `app.js` als Text parst.

Offen

- **Zwei Bestätigungs-Zustandsmaschinen bestehen weiterhin nebeneinander** (`voice.js:59-66` mit 15 s Frist, `app.js:2693-2836` mit 25 s Frist). Die erste läuft vor der zweiten und akzeptiert jedes `confirm` als Ja — genau die Lücke, die die zweite ausdrücklich schließen soll. Bei mittlerer Erkennungssicherheit fragt das System zweimal nach: drei Äußerungen für eine Buchung.
 - Die Rückfrage liest englische Intent-Bezeichner vor: `_ACTION_DE` deckt 6 von über 18 Aktionen ab, für den Rest sagt die Sprachausgabe wörtlich „Ich habe `confirm_all` verstanden. Richtig?“.
 - Der Sprachstatus-Indikator ist toter Code: das Element trägt ein inline gesetztes `display:none`, das nirgends entfernt wird, und es existiert keine zugehörige CSS-Regel. „Hört zu / Spricht / Unsicher“ erreicht den Nutzer nur über Toast und Ton.
 - Das Oberflächen-Mapping kennt sechs reale Ansichtsnamen nicht; sie melden dem Backend alle fälschlich eine Detailansicht.
 - Der erste LLM-Aufruf dauert 4255 ms bei einem konfigurierten Timeout von 4000 ms.
-

7. Tests und Qualitätssicherung — 70/100

Ebene	Tests	Lauffähig	Make-Ziel
Backend pytest	1086	ja, alle grün	<code>make test</code>
Backend live	24	nur mit Stack	keins
Odoo-Addons	198	nur im Container	keins
Infrastructure	130	ja	keins
PWA Node	47	ja, 47/47 grün in 185 ms	nur Teilziel
Playwright e2e	31–33	nein, Browser fehlt	nur 3 von 11 Specs
n8n Node	15	ja	keins

- **393 Tests haben kein Make-Ziel.** `make verify` ruft weder PWA- noch Infra-, Odoo- noch n8n-Tests auf. Das ist der größte Einzelhebel des Audits: vier Zeilen Makefile.
- Die Playwright-Fehlschläge sind **kein fachlicher Verfall**: `@playwright/test` verlangt Chromium-Build 1208, im Cache liegt 1228. Kein Test ist gescheitert, sie sind nie gestartet. Ein während dieses Audits begonnener, abgebrochener Installationslauf hat den Cache-Eintrag zusätzlich unvollständig zurückgelassen; ein einmaliges vollständiges `npx playwright install chromium chromium-headless-shell` stellt die Suite wieder her, sinnvollerweise unter Windows, wohin der Makefile-Pfad ohnehin zeigt.
- **Kein CI.** Kein `.github/`, keine Pipeline. Alles hängt an manueller Ausführung auf einer Windows-Maschine.
- Der Makefile ist unter WSL zu etwa einem Drittel unbenutzbar (`npm.cmd` , `npx.cmd`), und `verify-odoo-schema` ruft ein Fremdwerkzeug auf, das nicht installiert und nicht im Repository ist.

8. PWA und Bedienung — 68/100

Belegt

- 12 Ansichtszustände, vollständige Abläufe für Einzel- und Cluster-Kommissionierung, Serial-/Lot-Pflichteingabe, Kamera-Scanner, Qualitätsmeldung.
- Service Worker gepflegt (Cache `picking-v24` , 18 Assets), Update-Behandlung vorhanden, `/api/*` bewusst ausgenommen.
- Idempotenzschlüssel auf allen Schreibwegen.
- 47 Unit-Tests grün.

Offen

- **Kein Offline-Schreibweg.** Keine Outbox, kein IndexedDB, kein Background-Sync. Offline gebuchte Positionen enden in einem Fehler-Toast, die Arbeit ist verloren. Offline funktioniert nur die Anwendungshülle. Für eine Lager-PWA ist das die gewichtigste fachliche Lücke.
- Kontrast im hellen Modus: `#7c8ba1` auf `#f6f8fc` ergibt 3,26:1, WCAG AA verlangt 4,5:1 — 23 Verwendungen, überwiegend bei kleinen Schriftgraden.

- Sieben Bedienelemente liegen unter 44 px, im mobilen Modus bis hinunter zu 34 px, obwohl `--touch-min: 48px` definiert und Handschuhbedienung das erklärte Ziel ist.
 - Kein Zeitwächter auf Netzwerkaufrufen: bei hängender Verbindung bleibt die Oberfläche unbegrenzt im Ladezustand.
 - Verweigerter Kamerazugriff wird stumm verschluckt.
-

9. Sicherheit — 65/100

Belegt gut gelöst

- Sitzungstoken kryptografisch erzeugt, Cookies mit `Secure`, `HttpOnly` und `SameSite=strict`. Keine vom Client bestimmte Identität im Sitzungspfad.
- Anmeldedrosselung reserviert vor der Authentifizierung.
- Netzsegmentierung: `core-net` und `automation-net` sind `internal`.
- Body-Limit unterhalb der HMAC-Prüfung, als rohe ASGI-Middleware.
- Fehlende Geheimnisse führen zu 503, nie zu 500 und nie zu einer offenen Route.
- Der Edge lässt Dokumentationspfade und interne Routen mit 404 auflaufen, nicht 403.

Offen

- **Der laufende Stack fährt `RUNTIME_PROFILE=development`**. Im Container direkt nachgewiesen: `/api/docs`, `/api/openapi.json`, `/api/instances` und `/api/demo/traceability` antworten mit 200 ohne Authentifizierung. Nur Caddy verdeckt sie; alles auf dem Container-Netz sieht sie offen. Der gesamte Härtnachweis in `test_route_security.py` gilt ausschließlich für das Produktionsprofil und sagt über diese Deployment-Konfiguration nichts aus.
 - Keine Rollenautorisierung (siehe Bereich 1).
 - Eine Datei mit entschlüsselten n8n-Zugangsdaten liegt im Klartext außerhalb des Repositorys auf dem Desktop. Nicht in der Historie, aber beim Packen der Abgabe ausdrücklich auszuschließen.
 - Postgres läuft für alles über eine Superuser-Rolle.
 - Der Wechsel zwischen Lager 1 und Lager 2 verlangt kein Passwort — bewusst so entschieden, gehört aber ins Bedrohungsmodell der Arbeit.
-

10. Vorführbarkeit — 60/100

Sofort vorführbar am Rechner

Alle elf Container laufen. Zugriff über den Windows-Browser:

Dienst	Adresse
PWA	<code>https://localhost/</code>
n8n-Editor	<code>http://localhost:5678</code>
Odoo Lager 1	<code>http://localhost:8069</code>
Odoo Lager 2	<code>http://localhost:8070</code>
Backend	<code>http://localhost:8000</code>

Anmeldung in der PWA mit `lena.lager` oder `max.picker`, Passwort `admin`. Der Benutzer `admin` selbst hat keine Picker-Rolle und wird bewusst abgewiesen.

Der n8n-Editor ist **absichtlich nicht** über Caddy erreichbar: die entsprechenden Routen wurden entfernt, weil n8n sämtliche Workflow-Zugangsdaten hält. `https://localhost/n8n` liefert die PWA, nicht n8n.

Blockiert

- Vom Mobilgerät im WLAN ist keine sinnvolle Vorführung möglich: das Zertifikat kennt nur `localhost`, `127.0.0.1` und eine alte Hotspot-Adresse. Ohne gültiges HTTPS gibt es weder Mikrofonzugriff noch Service Worker — also weder Voice noch die PWA-Eigenschaft. `LAN_HOST` in der `.env` zeigt zudem auf eine nicht mehr vergebene Adresse.
- Die Bildbewertung lässt sich nur in Lager 1 zeigen.
- Der stärkste vorhandene End-to-End-Beleg (`e2e/ccluster.live.js` mit unabhängiger Nachprüfung in Odoo) hat ein gespeichertes Ergebnis vom 26. Juni — also aus der Fassung *vor* dieser Nachprüfung. Ein Neulauf schließt die Lücke.

11. Betrieb und Auslieferung — 55/100

- Kein CI, keine Pipeline, kein automatischer Testlauf.
- Kein Backup-Skript und kein Backup-Ziel im Makefile; die vorhandenen Dumps sind manuell erzeugt und stammen aus den Vorgängerdatenbanken.
- Zwei Runbooks vorhanden (Foundation-Rollout, n8n-Rollenmigration), keines für Backup, Wiederherstellung, Störfall oder Rückabwicklung.
- Das Backend-Image kann seine eigenen Tests nicht ausführen, weil es nur die Laufzeit-Abhängigkeiten installiert.
- Zertifikat und `LAN_HOST` sind veraltet.

12. Aufrufgraph-Analyse

Ergänzend zur bereichsweisen Prüfung wurde der Symbolgraph des Backends über einen Sprachserver ausgewertet (323 Symbole in `services/` und `routers/`, gegengeprüft über einen eigenen AST-Aufrufgraphen über 172 Python-Dateien). Für `pwa/js/app.js` stand kein JavaScript-Sprachserver zur Verfügung; die dortigen Aussagen sind textbasiert und damit schwächer belegt.

Ein ganzes Teilsystem ist unerreichbar geworden. `N8NWebhookClient` (`services/n8n_webhook.py`) hat zwei öffentliche Methoden, `fire_event:126` und `request_reply:169` — beide werden ausschließlich von Tests aufgerufen. Damit sind auch alle elf privaten Helfer der Klasse unerreichbar, einschließlich des vollständig implementierten Circuit-Breakers. Das Gerüst steht trotzdem noch: `runtime.py:127-134` baut den Client, `dependencies.py:61` liefert ihn aus, `PickingService` und `ClusterService` speichern ihn als `self._n8n` — und **kein einziger Lesezugriff auf `self._n8n` existiert**. Die direkte Webhook-Strecke wurde durch den Outbox-Weg ersetzt, aber nie ausgebaut. Dazu kommt `coerce_event_result:79`, das in zwei Services importiert, aber nirgends aufgerufen wird.

Vier Odoo-Methoden mit `api_`-Präfix sind verwaist, also als externe Schnittstelle gedacht, aber von Backend, n8n, PWA, Skripten und Odoo-XML nirgends aufgerufen: `api_check_login` und `api_record_login_result` (`auth_throttle.py:160,177` — durch das Begin/Finish-Paar ersetzt, ein Test assertiert sogar ausdrücklich, dass sie *nicht* mehr aufgerufen werden), `api_requeue_dead` (`outbox.py:212` — der Supervisor-Requeue ist implementiert und getestet, aber nirgends aufrufbar) und `api_get_job` (`integration_job.py:172` — existiert faktisch nur noch als Testwerkzeug).

Die Architektur hat genau einen Lastträger. `OdooClient.execute_kw` hat 59 direkte Aufrufstellen in 11 Dateien; jede Aufrufkette des Systems endet dort. Danach folgt in der Rangliste nicht Geschäftslogik, sondern Buchhaltung: Idempotenzverwaltung und Callback-Protokollierung. Auffällig sind drei Funktionen, die als Kopie in mehreren Routern liegen — `_finalize_error`, `_return_or_raise_replay` und `_m2o_id`, zusammen 42 Aufrufstellen für dreifach vorhandene Logik.

Zwei Fehlerbehandlungslücken auf den Kernpfaden, die die bereichsweise Prüfung nicht gefunden hat:

- Beim Bestätigen einer Position liegt der Odoo-Schreibvorgang (`picking_service.py:884`) *innerhalb* des Blocks, der bei einem Fehler die Idempotenz-Reservierung abbricht. Bricht Odoo nach dem Schreiben ab, wird die Reservierung verworfen, obwohl der Schreibvorgang bereits festgeschrieben ist — ein Wiederholungsversuch mit demselben Schlüssel läuft dann vollständig neu durch. Der Schreibvorgang selbst ist idempotent, der Effekt also begrenzt, aber die Reservierung schützt an dieser Stelle nicht das, was sie zu schützen vorgibt.
- Im Sprachpfad ist `finalize_external_intent` (`intent_engine.py:825`) die einzige Stelle des LLM-Abzweigs ohne Fehlerbehandlung. Eine Ausnahme dort wirft die gesamte Route auf 500, obwohl das deterministische Ergebnis bereits vorliegt — und widerspricht damit dem im Code ausdrücklich formulierten Vorsatz, dass jeder Fehler das deterministische Ergebnis erhält.

Positiv hervorzuheben: der Sprachpfad fängt an *jeder* Außengrenze breit ab — Whisper, Ollama, ffmpeg, Enum-Auswertung, JSON-Auswertung. Kein einzelner Ausfall bricht die Route. Eine Einschränkung bleibt: scheitert die Audiokonvertierung, werden stillschweigend die unkonvertierten Originaldaten weitergereicht; von außen ist das nicht von „nichts gesagt“ zu unterscheiden, nur das Protokoll trennt die beiden Fälle.

`pwa/js/app.js` ist zerlegbar. 4275 Zeilen, 167 Funktionen auf oberster Ebene, keine davon tot. Die Kopplungsanalyse zeigt drei saubere Schnitte: 23 DOM-Zugriffs- Funktionen ohne jede Auswärtskopplung, acht gemeinsam genutzte Helfer, und der Cluster-Block über rund 770 Zeilen mit nur zwei Einstiegspunkten von außen. Zusammen etwa ein Viertel der Datei bei sehr wenigen anzupassenden Aufrufstellen.

13. Ordnerstruktur — was bleibt und was weg kann

Auf der Platte liegen rund 615 MB unter `Bachelor/` und weitere 26 MB unter `Bachelor-wt/`. Versioniert sind davon 588 Dateien. Das Repository ist also sauber — der Ballast liegt daneben.

Muss bleiben

Pfad	Warum
<code>Mobile Picking und Voice Assistant/</code> (ohne die unten genannten Unterordner)	Das Projekt selbst
<code>Projekt-Wiki/</code>	80 versionierte Dateien, Kapitel 00–12 der Arbeit
<code>infrastructure/bildkorpus/</code>	Messskripte und Messwerte des empirischen Teils
<code>docs/</code> einschließlich <code>architecture/</code>	Die 13 Architekturebenen mit Diagrammen
<code>docs/screenshots/</code> , <code>docs/testing/</code>	Abbildungen der Arbeit — derzeit weder versioniert noch ignoriert , also ungeschützt
<code>infrastructure/backups/</code> (Satz vom 31. Juli)	Letzter Stand vor dem Cutover; als geschlossener Satz aufbewahren
<code>n8n/backups/20260807-121632/</code>	Jüngster vollständiger n8n-Stand
<code>.git/</code>	Historie
Eigenständigkeitserklärung (PDF)	Pflichtdokument

Kann ohne Verlust weg

Pfad	Größe	Warum unbedenklich
<code>graphify/</code>	339 MB	Fremdes Analysewerkzeug mit eigener Versionsverwaltung, seit 1. Juli unberührt, kein Projektbestandteil
<code>backend/.deps/</code>	87 MB	Nachinstallierbare Bibliotheken. Achtung: danach ist <code>make test</code> defekt, bis <code>make</code> einmal neu läuft
<code>n8n/custom-nodes/.../node_modules/</code>	73 MB	Über <code>npm install</code> wiederherstellbar
<code>graphify-out/</code> (zwei Kopien)	18 MB	Erzeugter Analyseausgabe, nirgends referenziert
<code>node_modules/</code> (Projektwurzel)	17 MB	Wiederherstellbar
<code>.claude/</code> , <code>.serena/</code> , <code>.superpowers/</code> , <code>.design/</code>	~20 MB	Werkzeugverzeichnisse der KI-Assistenz
<code>test-results/</code> , <code>.pytest_cache/</code> , <code>n8n/tmp/</code>	~1 MB	Erzeugte Zwischenstände
<code>~/odoo19-migration-backups/</code>	9,4 MB	Sicherung vom 4. Juli, durch die vom 31. Juli überholt
10 von 11 Ständen in <code>n8n/backups/</code>	0,4 MB	Zwischenstände aus vier Aufbau Tagen, drei davon leer
<code>Notzien/</code>	0 Byte	Leerer Ordner, Fehlanlage
<code>odoo/addons/picking_assistant_context/</code>	0 Byte	Nur leere Unterordner ohne Manifest, täuscht ein viertes Modul vor

Zusammen rund **460 MB ohne jedes Risiko**, mit den Bibliotheken etwa **545 MB** — gut 85 Prozent der Belegung, ohne eine einzige Datei zu berühren, die in die Abgabe gehört.

Gesondert zu behandeln

Der Ordner `backup-n8n-2026-08-04/` enthält neben einer nützlichen Sicherung eine Datei mit **entschlüsselten Zugangsdaten im Klartext**. Sie liegt außerhalb der Versionsverwaltung und ist damit nicht in der Historie, wandert aber bei jedem unbedachten Kopieren oder Packen mit. Sie gehört verschlüsselt und getrennt abgelegt.

Versionsverwaltung

`main` steht 261 Commits hinter dem Arbeitsbranch und ist dessen direkter Vorfahre — ein Fast-Forward ohne Konfliktmöglichkeit. 15 Branches sind eingeflossen und löscherbar, drei Arbeitsverzeichnisse (`Bachelor-wt/`) sind funktionslos. Zwei Zweige bleiben bewusst stehen: `remediation/r4-postgres` (die abgelehnte Rollentrennung, als dokumentierte Entscheidung wertvoll) und `docs/projektdoku` (noch nicht eingeflossene Dokumentationsarbeit, vorher inhaltlich prüfen).

Was jetzt zu tun ist

Stand nach der Abarbeitung am 19. August: erledigt sind 1 bis 6, 8 und 9, Punkt 6 durch Verdrahtung statt durch Dokumentation. Punkt 10 ist zur Hälfte erledigt — die Make-Ziele gibt es, Chromium wurde bewusst nicht installiert, weil Playwright entfernt wurde. Punkt 7 entfällt aus demselben Grund; an seine Stelle tritt der an diesem Tag live gefahrene Nachweis über Anmeldung, Auftragsliste und die vollständige KI-Bildbewertung. Offen bleiben 11 bis 17 sowie zwei bewusst getroffene Entscheidungen: kein Offline-Schreiben und keine Postgres-Rollentrennung vor der Abgabe. Einzelheiten im Nachtrag.

Vor jeder Vorführung und vor der Abgabe

1. `main` per Fast-Forward nachziehen und pushen. Ohne Konfliktrisiko, `main` ist direkter Vorfahre. Andernfalls sieht jeder Prüfer den Stand vom 8. Juli.
2. `docs/screenshots/` und `docs/testing/` committen, bevor irgendein Aufräumen beginnt — sonst vernichtet ein Standard-Cleanup Abbildungen der Arbeit.
3. Die Klartext-Credentials-Datei aus dem Desktop-Ordner herausnehmen und verschlüsselt ablegen.
4. Zertifikat mit der aktuellen WLAN-Adresse neu ausstellen und `LAN_HOST` setzen, falls am Mobilgerät vorgeführt werden soll. Ohne das kein Voice und keine PWA.
5. Entscheiden, wie das Deployment-Profil behandelt wird: entweder den Stack einmal mit `RUNTIME_PROFILE=production` starten und den Härtungsnachweis dagegen führen, oder in der Arbeit ausdrücklich als Entwicklungs-Deployment kennzeichnen. Beides ist vertretbar, das Schweigen darüber nicht.

Für die inhaltliche Belastbarkeit der Arbeit

1. Rollenautorisierung verdrahten oder als bewusste Systemgrenze dokumentieren. Eine Arbeit, die Rollen und Rechte als eigene Architekturebene führt, sollte diese Lücke nicht offenlassen.
2. `e2e/c1uster.live.js` neu laufen lassen, damit der stärkste Systembeleg mit der Odoo-Nachprüfung erzeugt wird. Das ist der zitierfähige End-to-End-Nachweis.
3. Fehlerbenachrichtigung in `n8n` reparieren (Header über eine Credential statt über `$env`). Ein System, dessen Fehlerweg zu 100 Prozent scheitert, hat keine Fehlerbehandlung.
4. Backup der produktiven Odoo-19-Datenbanken erstellen, mit Prüfsummen.
5. Vier Make-Ziele ergänzen, damit die 393 verwaisten Tests überhaupt laufen, und Chromium in der passenden Version installieren.

Aufräumen, sobald Zeit ist

1. Zwei Voice-Zustandsmaschinen zusammenführen, `_ACTION_DE` vervollständigen, Statusanzeige sichtbar machen.
2. Kontrastwert und Bedienelementgrößen korrigieren.
3. Tote v1-Routen und -Webhookziele ausbauen, ungenutzte Modelle entfernen.
4. Überholte Vorbehalte in Ebene 1 und 5 streichen, Scorecards für Ebene 12 und 13 nachziehen, Testzahlen aktualisieren.
5. Rund 460 MB Ballast entfernen, gemergte Branches und Worktrees aufräumen.
6. `N8NWebhookClient` samt Durchreichung durch beide Services ausbauen, die vier verwaisten `api_`-Methoden in Odoo entfernen oder wieder anbinden, und die drei dreifach kopierten Router-Helfer

zusammenführen.

7. Die beiden Fehlerbehandlungslücken schließen: Schreibvorgang aus dem Abbruchbereich der Idempotenz-Reservierung herausnehmen, `finalize_external_intent` absichern.
-

Was diese Arbeit belegen kann — und was nicht

Belastbar belegt:

- Die KI-gestützte Qualitätsbewertung funktioniert Ende zu Ende, 40 von 40 Läufen erfolgreich, mit Bildauswertung, in 66 bis 147 Sekunden.
- Das Latenzproblem der Spracherkennung ist gelöst und nachgemessen.
- Der Zwei-Lager-Betrieb ist driftfrei und funktioniert.
- Die Backend-Sicherheitsarchitektur ist im Produktionsprofil durch 1086 Tests abgesichert, darunter ein Test, der die echte Routentabelle prüft.
- Die Architekturdokumentation stimmt nachweisbar mit dem Code überein.

Nicht belegt, ehrlich zu benennen:

- Kein Nachweis über das produktive Deployment-Profil.
- Keine Rollenauthorisierung im Backend.
- Kein Offline-Betrieb für Schreibvorgänge.
- Keine automatisierte Qualitätssicherung.
- Keine Erkennungsrate mit menschlicher Stimme unter realen Lagerbedingungen.
- Keine Nebenläufigkeitsprüfung unter Last.

Diese Liste ist kein Mangel der Arbeit, solange sie in der Arbeit steht. Eine Abschlussarbeit, die ihre Grenzen benennt, ist stärker als eine, die sie verschweigt.